▶ 1분 미리보기

Chapter 4. Claude Code 확장하기

나의 Claude Code를 팀의 플랫폼으로 (2026.11, Claude Code v2.1.319, Bedrock 기준)

Huh, Jinsung
Sr. Solutions Architect
AWS Korea

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

기대 주제 2위, 팀 표준화

사전 설문 결과로 정한 이 챕터의 목표

PRE-SURVEY

58명

참가 확정자 **138명 중 58명**이 팀 표준화(설정, 권한, 훅, MCP)를 기대 주제로 골랐습니다.

이 챕터에서 그 표준을 플러그인 하나로 묶어 배포합니다.



기대 주제 2위, 복수 선택 문항

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

강의 35분 + 랩 55분 (선택 12분)

세 개의 Part와 Super Lab, 개인 설정에서 팀의 플랫폼까지

PART 01 / 12분

Plugins

여섯 가지 확장 방법, 플러그인 구조, 설치

PART 02 / 10분

사내 배포

마켓플레이스, 강제 설치, 망분리

PART 03 / 13분

Mods

Claude Code 안에서 도는 코드

SUPER LAB / 55분

팀 스타터 킷 + mod

플러그인 하나로 묶기

MORE

더 알아보기

폴더 구성, 마켓플레이스 소스, 조직 배포 설정, mod 이벤트와 호출, mod 리뷰, 운영 지표

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CHAPTER 04

Claude Code 확장하기

Plugins & Mods로 스킬, Subagent, 훅, MCP, mod를 한 번에 배포합니다.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CHAPTER 04 / PART 01

Plugins

팀의 작업 방식을 묶어 배포하는 단위

01
여섯 가지 확장 방법

02
플러그인 구조

03
설치와 갱신

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

여섯 가지 중 다섯을 한 폴더에

무엇을 넣을지는 언제, 누가, 무엇을 바꾸는가로 선택

01
CLAUDE.md
플러그인 밖

PLUGIN, 한 디렉터리로 배포

02
스킬

03
Subagent

04
훅

05
MCP

06
mod

CLAUDE.md는 managed CLAUDE.md나
저장소로,
나머지 5종은 한 디렉터리에 담아
플러그인으로 배포합니다.
하나를 누르면 언제 쓰는지 나옵니다.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

폴더 하나 = 팀의 표준

플러그인 디렉터리 구성과 각 파일의 역할

asset-team-kit/

`.claude-plugin/plugin.json`	이름, 버전, 설명
`skills/retirement-gap/SKILL.md`	연금 격차 절차
`agents/spending-analyst.md`	Ch3에서 만든 것

> `agents/portfolio-analyst.md, risk-reviewer.md`
> Ch3 3종

`hooks/hooks.json`	DLP 훅 등록 (Ch2)
`scripts/dlp-guard.sh`	검사 스크립트
`commands/*.md`	슬래시 커맨드
`.mcp.json`	MCP 서버

점선 두 줄은 이 예시에는 없지만 플러그인에 넣을 수 있는 파일입니다.

파일을 누르면 무엇이 들어가는지 보여 줍니다. Ch2와 Ch3의 결과물을 그대로 넣고, 설치 한 줄로 동료가 같은 환경을 갖습니다.

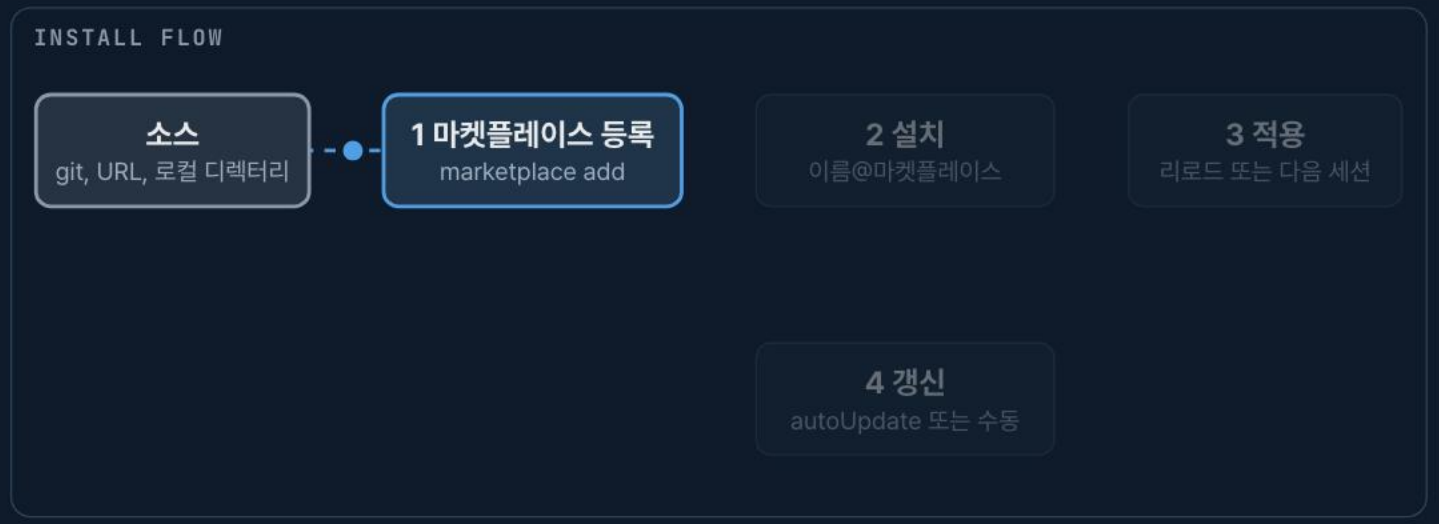
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

등록, 설치, 적용, 갱신의 네 단계

마켓플레이스에서 동료의 세션까지 플러그인이 가는 길

STEP 1 **마켓플레이스를 등록합니다**

플러그인은 **마켓플레이스**를 통해 받습니다. 먼저 **/plugin marketplace add**로 마켓플레이스를 등록합니다. 소스는 git, URL, 로컬 디렉터리입니다.



● 등록 ● 설치 ● 적용 ● 갱신

마켓플레이스 등록

Claude Code 프롬프트, 소스별 예시

```
# 로컬 경로 (이 랩)
/plugin marketplace add ~/claude-lab/ch4/ma
rketplace

# GitHub 저장소
/plugin marketplace add owner/repo

# 사내 git URL
/plugin marketplace add https://git.corp.ex
ample/...git
```

망분리 환경은 등록 대신 **seed 디렉터리**를 씁니다 (Part 2).

- 로컬 경로: 개발, 테스트 (이 랩)
- GitHub 저장소: 공개, 사내 GitHub
- git URL: 사내 GitLab 등

‹ 1 / 5 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CHAPTER 04 / PART 02

사내 배포

설치를 강제하고 출처를 제한한다

01
사내 마켓플레이스 강제

02
망분리와 CI

03
배포 후 측정

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

허용, 등록, 강제, 사이드로드 금지

managed-settings.json 네 개 키로 사내 마켓플레이스를 강제

OVERVIEW **키 네 개가 각자 한 가지를 합니다**

사내 마켓플레이스를 강제하려면 **허용 목록, 등록, 강제 설치, 사이드로드 금지**를 함께 겁니다. 한 키만으로는 다 막히지 않습니다.



사내 마켓플레이스 강제

managed-settings.json

```
{
 "strictKnownMarketplaces": [
  { "source": "github", "repo": "your-org/*" },
  { "source": "hostPattern",
    "hostPattern": "^git\\.corp\\.example$" },
  { "source": "skills-dir" } ],
 "extraKnownMarketplaces": {
  "fsi-tools": { "source": { "source": "github",
    "repo": "your-org/fsi-tools" } } },
 "enabledPlugins": {
  "asset-team-kit@fsi-tools": true },
 "disableSideloadFlags": true
}
```

네 줄이 각각 허용, 등록, 강제, 금지입니다. 다음 단계에서 하나씩 봅니다.

● 관리자 정책 ● 허용과 등록 ● 강제 설치 ● 거부되는 경로

‹ 1 / 5 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

허용 목록이면 사이드로드도 막힐까?

QUICK CHECK, 손을 들어 답해 주세요

strictKnownMarketplaces 허용 목록만 설정하면 `--plugin-dir` 로컬 로드는?

A 막힌다

B 막히지 않는다

C 경고만 뜬다

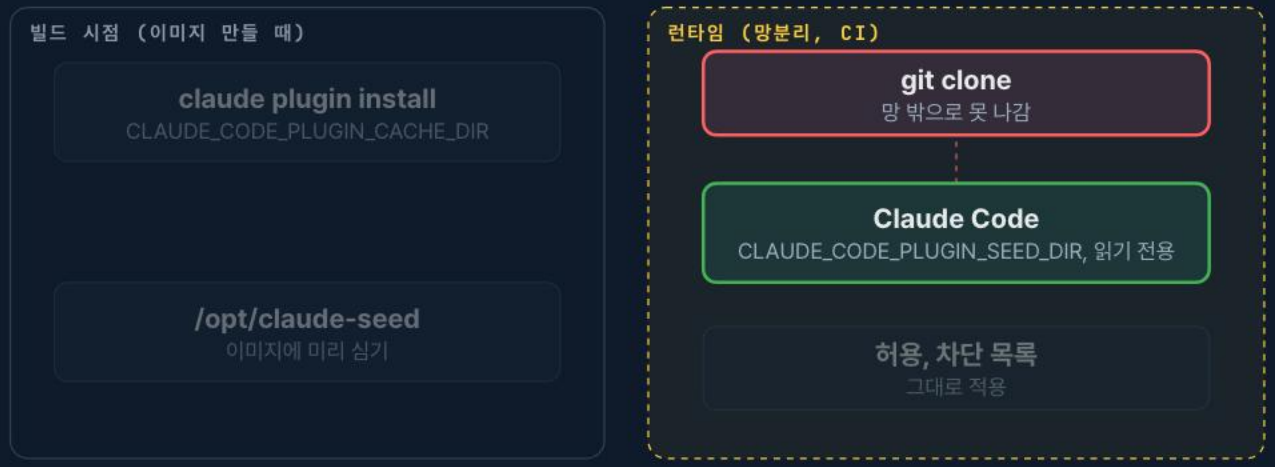
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

git clone 불가 환경엔 사전 배치

망분리와 CI는 CLAUDE_CODE_PLUGIN_SEED_DIR

STEP 1 **망분리 환경은 git clone이 안 됩니다**

마켓플레이스 등록과 설치는 보통 git이나 URL에서 받아 옵니다.
망분리 PC나 CI(자동 빌드와 테스트) 러너에서는 그 길이 막혀 있습니다.



망분리, CI

플러그인을 받는 방법

```
# 빌드 시점
CLAUDE_CODE_PLUGIN_CACHE_DIR=/opt/claude-seed \
  claude plugin install …

# 런타임
CLAUDE_CODE_PLUGIN_SEED_DIR=/opt/claude-seed
```

해법은 **빌드할 때 심고, 런타임에는 읽기만** 하는 것입니다.

- 망분리 업무 PC
- 외부 접근이 없는 CI 러너
- 설치할 때 받아 오는 방식이 안 맞음

● 설치와 정책 ● seed 디렉터리 ● 런타임 사용 ● 막힌 경로

‹ 1 / 4 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

설치 다음은 사용 확인

배포 후 측정, OTel 이벤트 두 개와 이름 가림

EVENT

plugin_installed

설치마다 기록

EVENT

plugin_loaded

세션 시작 시 켜진 플러그인

PRIVACY

이름 가림

서드파티 플러그인 이름

무엇이 설치되고 쓰이는지 OTel로 봅니다. 카드를 누르면 각 이벤트가 답하는 질문이 나옵니다.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CHAPTER 04 / PART 03

Mods

Claude Code 안에서 도는 코드

01
mod가 할 수 있는 것

02
구조와 예시

03
조직 통제

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

NEW, V2.1.287+

mod = 플러그인 안의 이벤트 핸들러

도구 호출, 프롬프트, 커맨드, 화면 그리기 이벤트를 관찰하고, 바꾸고, 대신 처리합니다. Bedrock에서도 동작합니다.

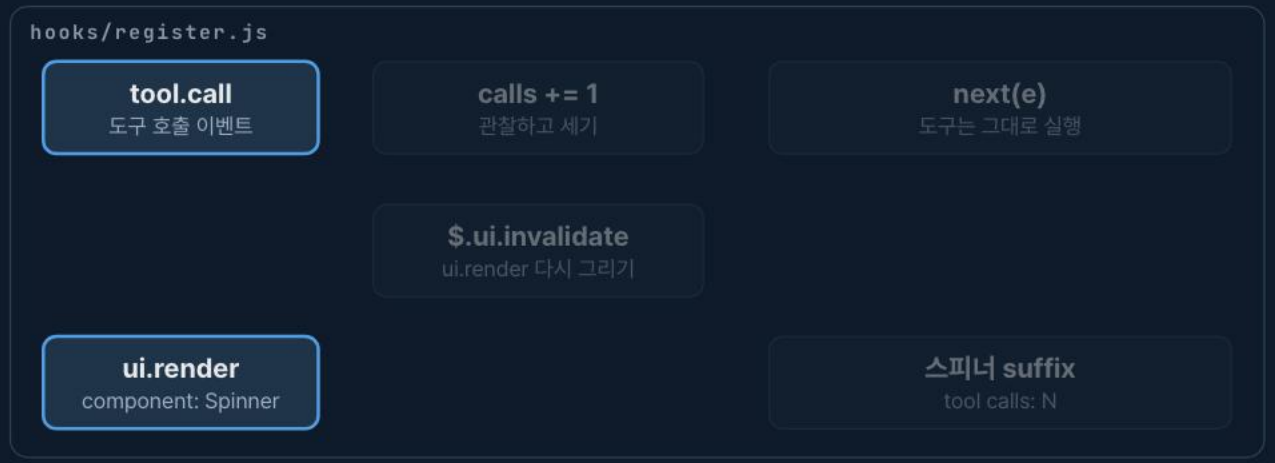
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

스피너 옆 도구 호출 수

가장 작은 mod, 이벤트 두 개를 다루는 register.js

STEP 1 **register(on)으로 핸들러를 겁니다**

mod는 **register 함수** 하나를 내보냅니다. Claude Code가 넘겨준 **on**으로 어떤 이벤트에 끼어들지 등록합니다.



가장 작은 mod

hooks/register.js

```
let calls = 0
export function register(on) {
  on('tool.call', async ($, e, next) => {
    calls += 1
    $.ui.invalidate('ui.render')
    return next(e)            // 도구는 그대로 실행
  })
  on('ui.render', { component: 'Spinner' }, async
($, e, next) => {
    return next({ ...e, props: { ...e.props,
      suffix: ' · tool calls: ' + calls + '…' } })
  })
}
```

핸들러는 `($, e, next)` 를 받습니다. $는 바깥 작업용 API입니다.

● 이벤트 ● 핸들러 동작 ● 결과

‹ 1 / 5 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

mod는 최후의 수단

mod와 다른 확장 비교, 바꿀 수 있는 것과 작성 언어

JS/TS
mod

스크립트
Settings 훅

Markdown
Skill

아무 언어
MCP

네 가지를 눌러 비교합니다. 원칙은 하나, **이미 다른 수단으로 되면 mod를 쓰지 않습니다**.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

공식 샘플 셋으로 감 잡기

claude-code-playground 저장소, 공식 지원 대상은 아님

CONTEXT
token-weather

SAFETY
blast-radius

REVIEW
replay-theater

샘플 하나를 누르면 무엇을 하는 mod인지 보여 줍니다.

샘플은 claude-code-playground 저장소에 있으며 공식 지원 대상이 아닙니다. 도입 전에 더 알아보기의 mod 리뷰 절차를 거칩니다.

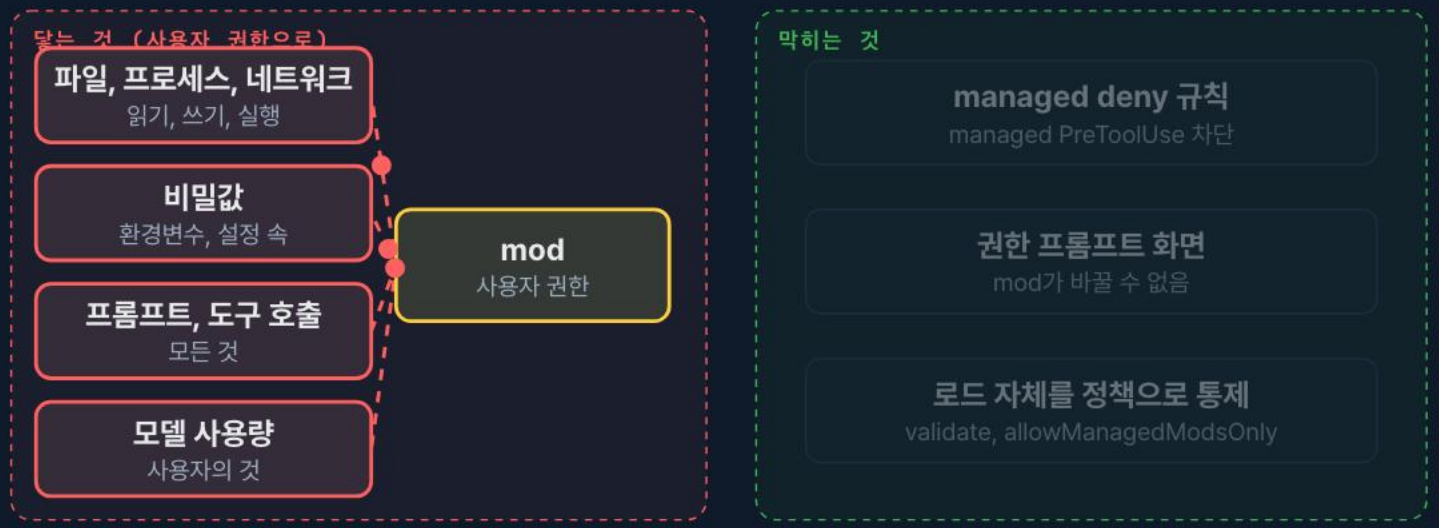
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

mod의 범위 = 사용자 권한 전부

설치 전에 알아야 할 신뢰 모델, 닿는 것과 막히는 것

STEP 1 **사용자가 닿는 곳엔 mod도 닿습니다**

mod는 **사용자 권한으로** 돕니다. 파일 읽기와 쓰기, 프로세스 실행, 네트워크, 환경변수와 설정 속 비밀값, 모든 프롬프트와 도구 호출, 모델 사용량까지 닿습니다.



닿는 것

사용자 권한으로

- 파일 읽기, 쓰기
- 프로세스 실행
- 네트워크
- 환경변수, 설정 속 비밀값
- 모든 프롬프트와 도구 호출
- 사용자의 모델 사용량

그래서 **남이 만든 mod**는 플러그인보다 더 엄격하게 검토합니다.

- mod에는 샌드박스가 없음
- 비밀값과 모든 프롬프트에 접근
- 설치는 곧 신뢰

● mod가 닿는 것 ● mod가 못 넘는 것 ● 로드 통제

‹ 1 / 3 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

사용자 mod는 막고, 조직 mod는 검토

allowManagedModsOnly와 claude plugin validate 읽는 법

STEP 1 **allowManagedModsOnly: 조직 mod만 로드**

내장 보안 플러그인 `cc-plugin-sec-default@builtin`의 옵션 **allowManagedModsOnly**를 켜면 **조직이 배포한 mod만** 로드됩니다.



● 관리자 정책 ● 검토 ● 허용된 mod ● 막힌 mod

사용자 mod 차단

`managed-settings.json`

```
{
  "pluginConfigs": {
    "cc-plugin-sec-default@builtin": {
      "options": { "allowManagedModsOnly": true }
    }
  }
}
```

Super Lab TASK 07에서 이 설정으로 사용자 mod가 막히는지 확인한 뒤 원복합니다.

- managed-settings.json의 pluginConfigs
- 사용자가 설치한 mod는 로드 안 됨
- 사이드로드는 disableSideloadFlags와 함께

‹ 1 / 4 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

managed settings 없으면 guard는?

QUICK CHECK, 손을 들어 답해 주세요

Bedrock 사용자 PC에 managed settings가 없다면, 내장 guard(sec-default)는?

A 항상 로드된다

B 로드되지 않는다

C Bedrock이면 자동 로드

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

--plugin-dir로 실은 mod를 막으려면?

QUICK CHECK, 손을 들어 답해 주세요

사용자가 `--plugin-dir`로 mod를 실어도 로드되지 않게 하려면?

A CLAUDE.md에 금지 문구

B managed settings의 disableSideloadFlags와 allowManagedModsOnly

C 사용자 settings.json에 enabledPlugins false

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

약 60분, 팀 스타터 킷 제작

SUPER LAB, 팀 스타터 킷 플러그인 + mod

TASK 01-02	retirement-gap 스킬 만들기, Ch3 에이전트 3종 + DLP 훅과 asset-team-kit으로 묶기	Part 1
TASK 03-04	fsi-local에서 asset-team-kit@fsi-local 설치, 킷으로 pension-product.md 완성	Part 1
TASK 05	공식 first-mod 만들기, Claude에게 /tally-reset 추가 요청	Part 3
TASK 07	allowManagedModsOnly로 사용자 mod 차단 확인, 원복	Part 3
TASK 08	배포 전 체크리스트 (선택 TASK 06: Bedrock용 idle compaction mod, 12분)	Part 2

TASK 00에서 Ch3 결과물을 확인하고, 없으면 키트 기준본으로 채웁니다. 골격은 복사해 붙이고, 나머지는 Claude에게 시킵니다.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

배포 전 여섯 가지 점검

사내 배포 전 체크리스트, 눌러서 하나씩 확인

○ VALIDATE

validate 통과

hooks, calls 목록을 보안팀과 리뷰

○ VERSION

버전 고정

release channel(stable, latest) 분리

○ ALLOW LIST

허용 목록

strictKnownMarketplaces + disableSideloadFlags

○ AIR-GAP

망분리 경로

필요하면 seed 디렉터리로 배포

○ MEASURE

측정

plugin_loaded 이벤트로 사용 현황 확인

○ ROLLBACK

롤백

enabledPlugins false로 즉시 차단

Super Lab TASK 08에서 같은 목록을 직접 점검합니다.

0 / 6

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CHAPTER 04 / DEEP DIVE

더 알아보기

자주 받는 질문과 세부 설정

01
플러그인 폴더 구성 (Part 1 구조 장표)

02
마켓플레이스 소스

03
조직 배포 설정

04
mod 이벤트와 호출

05
남이 만든 mod 리뷰

06
플러그인 운영 지표

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

마켓플레이스의 네 출처

더 알아보기, /plugin marketplace add로 등록하는 소스

로컬 경로
개발, 테스트

GitHub 저장소
공개, 사내 GitHub

git URL
사내 GitLab 등

seed 디렉터리
망분리

소스를 누르면 예시와 맞는 경우가 나옵니다.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

사내 git 등록과 강제

더 알아보기, 조직 배포 설정과 함께 거는 잠금

managed-settings.json

```
{
  "extraKnownMarketplaces": {
    "fsi-internal": {
      "source": { "source": "git", "url":
        "https://git.corp.example/claude/marketplace.git" }
    }
  },
  "enabledPlugins": {
    "asset-team-kit@fsi-internal": true
  }
}
```

strictKnownMarketplaces
hostPattern으로 사내 git 허용

disableSideloadFlags
--plugin-dir 금지

allowManagedModsOnly
사용자 mod 차단

문제가 생기면
enabledPlugins를 false로

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

남의 mod, 샌드박스 없음

더 알아보기, validate 두 줄 읽기와 리뷰 여섯 항목

validate 두 줄 읽기 | **남이 만든 mod 리뷰**

항목	예 (FIRST-MOD)	읽는 법
hooks:	session.start, tool.call	언제 끼어드는가
hooks:	command.run{tally}, ui.render{Spinner}	커맨드, 화면 변경
calls:	$.command.register, $.ui.invalidate	바깥에 하는 일
위험 신호	프로세스 실행, 네트워크 호출	calls 줄에 보이면 보안 리뷰

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

배포 후 확인할 네 가지

더 알아보기, 플러그인 운영 지표

plugin_installed

OTel 이벤트

누가 설치했나

plugin_loaded

OTel 이벤트

실제로 쓰이나

스킬, 에이전트 호출

skill_activated, subagent_completed 이벤트

어떤 구성요소가 효과적인가

버전 분포

설치 캐시, 이벤트 속성

구버전이 남아 있나

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

팀의 플랫폼을 만드는 여섯 줄

Chapter 4 Key Takeaways

01

플러그인 = 스킬 + Subagent + 훅 + MCP + mod

02

조직 배포 = 허용 목록 + 등록 + 강제 설치 + 사이드로드 금지

03

망분리는 `CLAUDE_CODE_PLUGIN_SEED_DIR`

04

mod는 Claude Code 안의 JS 핸들러, 화면까지 바꿈

05

mod는 샌드박스가 없다. validate와 allowManagedModsOnly

06

Bedrock 사용자는 managed settings가 있어야 guard 로드

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Thank You

다음은 점심, 그리고 Chapter 5. 비용 효율적으로 쓰기입니다. 질문은 Slack #20261116-claude-code-workshop-for-fsi 채널에서 언제든 받습니다.

← 목차로 돌아가기 Chapter 5 →

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.